EXPERIMENT 5

Aim

To apply advanced GitHub workflows including a branching strategy, protected branches, code review, and automated CI using GitHub Actions.

Theory

A branching strategy defines how teams create and merge branches to avoid chaos. Common models include Git Flow (long-lived develop/release branches) and the simpler GitHub Flow (short-lived feature branches merged into main via PRs). Feature branches isolate work-in progress so main always stays deployable.

Branch protection rules on GitHub enforce quality gates: requiring PR reviews, passing status checks and up-to-date branches before merging. This prevents unreviewed or broken code from reaching main.

GitHub Actions is a CI/CD platform that runs workflows defined in YAML files under .github/workflows/. A workflow is triggered by events (push, pull_request) and runs jobs (e.g. install deps, run tests, lint) on hosted runners, automating quality checks on every change.

Software / Tools Required

Git, GitHub account, a Python project with tests, YAML editor.

Procedure (Step-by-Step)

Step 1. Adopt GitHub Flow: create a short-lived feature branch off main.

```
git checkout main && git pull
git checkout -b feature/add-calculator
```

Step 2. Add a small tested function to demonstrate CI.

```
# src/calc.py
def add(a, b):
 return a + b

# tests/test_calc.py
from src.calc import add
def test_add():
 assert add(2, 3) == 5
```

Step 3. Create a GitHub Actions workflow that runs tests on every push and PR.

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
```

```
test:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: actions/setup-python@v5
with:
python-version: "3.11"
- run: pip install pytest
- run: pytest -q
```

Step 4. Commit and push the branch to trigger the workflow.

```
git add .
git commit -m "Add calculator with CI workflow"
git push -u origin feature/add-calculator
```

Step 5. Open a Pull Request and watch the Actions check run under the 'Checks' tab. #

```
GitHub UI: the CI job runs automatically on the PR
```

Step 6. On GitHub, add a branch protection rule for 'main' requiring a passing CI check and one review.

```
# Settings -> Branches -> Add rule -> require status checks + reviews
```

Step 7. Request a review, address comments, and merge only after the check is green. #

```
Merge is blocked until CI passes and review is approved
```

Step 8. Use a status badge in the README to show build health.

```
![CI](https://github.com/username/repo/actions/workflows/ci.yml/badge.svg)
```

Step 9. Add a linting job to the same workflow for stricter quality gates.

```
- run: pip install flake8
- run: flake8 src/ --max-line-length=100
```

Step 10. Delete the merged feature branch to keep the repo tidy.

```
git branch -d feature/add-calculator
git push origin --delete feature/add-calculator
```

Expected Output

Each push shows a green CI check on GitHub. Branch protection blocks merging until tests pass and the PR is approved, demonstrating an automated, review-gated workflow.

Conclusion

An advanced GitHub workflow with feature branches, protected main, code review and automated CI via GitHub Actions was implemented, mirroring professional software delivery pipelines.

Viva Questions

1. Compare Git Flow and GitHub Flow.
2. What are branch protection rules and why are they used?
3. What triggers a GitHub Actions workflow?
4. What is Continuous Integration and what problem does it solve?
5. What is the role of a hosted runner in GitHub Actions?